

OPERATING SYSTEM CONCEPTS

Chapter 2: Operating-System Structures

Topic 1: Spooling

1. Definition

Spooling stands for **Simultaneous Peripheral Operations OnLine**.

It is the process of placing data of various I/O jobs into a **buffer** (a special area in memory or disk) so that slower I/O devices can access data at their own speed without making the CPU wait.

The OS manages this buffer to allow the CPU to continue executing other tasks while I/O is pending.

2. Core Concept Explanation

Different I/O devices operate at very different speeds. For example, a printer is much slower than the CPU. Without spooling, the CPU would have to wait idle until the printer finishes.

Spooling solves this by:

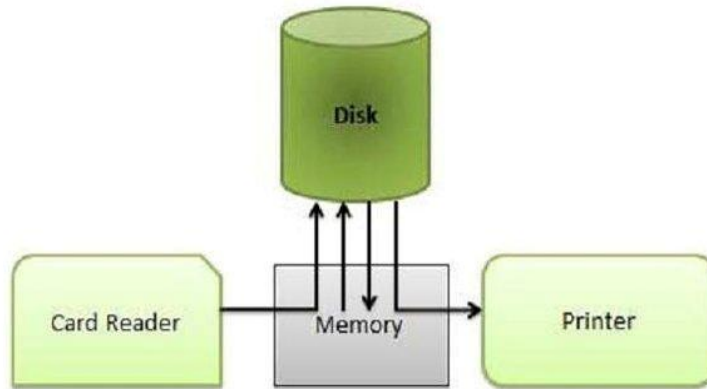
- **Collecting** the output data in a buffer (disk or RAM).
- Allowing the **CPU to continue** working on other tasks.
- Having the I/O device read from the buffer **at its own speed**.

This allows multiple I/O jobs to be handled **simultaneously and in parallel**.

3. Working Process (Step-by-Step)

1. **Job arrives:** A program sends data intended for an I/O device (e.g., printer).
2. **Data goes to buffer:** OS stores the data in the spooling buffer on disk, not directly to the device.
3. **CPU is freed:** The CPU does not wait; it moves on to the next task.
4. **OS scheduler:** The OS scheduler manages the queue of pending I/O jobs in the buffer.
5. **Device reads buffer:** The slow I/O device reads data from the buffer at its own pace.
6. **Job completes:** Once the device finishes, the OS removes that job from the buffer.

4. Diagram



The Memory/Buffer acts as a central waiting station. The Card Reader sends data in, the Disk stores it, and the Printer reads it at its own speed.

5. Practical Example

Real-life: When you print 10 documents in a university computer lab, all 10 are placed in a print queue (the spool). The printer picks them one by one. You do not have to wait at the computer until all printing is done.

System: On Linux, the CUPS (Common Unix Printing System) uses spooling. All print jobs are stored in `/var/spool/cups/` and sent to the printer sequentially.

6. Key Points

- **SPOOL** = Simultaneous Peripheral Operations OnLine.
- Spooling uses a **buffer in memory or hard disk** to store I/O data.
- OS maintains the spooling buffer and controls device access.
- Enables **parallel computation**: CPU works while I/O happens simultaneously.
- The most common example is **printer spooling**.

7. Advantages and Disadvantages

Advantages	Disadvantages
CPU utilization improves as it does not wait for slow devices.	Requires extra disk space for the spooling buffer.
Multiple jobs can be queued and handled in order.	If the buffer fills up, new jobs must wait.

Parallel I/O operations become possible.

Managing the spool adds OS overhead.

8. Short Revision Notes

Spooling = buffering data for slow I/O devices so the CPU can continue working. OS stores data in a spool buffer (disk/memory). The device reads from the buffer at its own speed. Most familiar example is the printer queue. Key benefit: parallel I/O + CPU execution.

Topic 2: Operating System Services

1. Definition

Operating System Services are a **set of functions** provided by the OS to help users and application programs run efficiently and correctly.

These services form the environment in which programs execute. They act as a bridge between user programs and the hardware.

2. Core Concept Explanation

OS services are divided into two categories:

Category A: Services for the User — helpful functions that users and programs directly benefit from.

Category B: Services for System Efficiency — internal functions to manage resources and security.

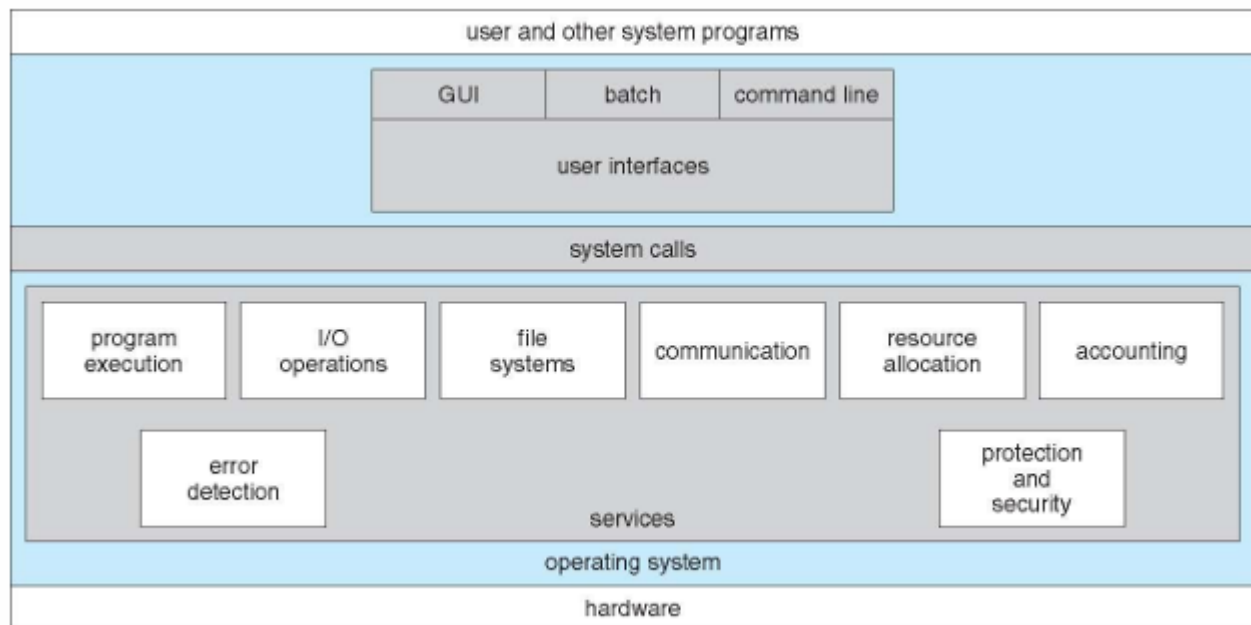
3. Category A: User-Helpful Services

- **User Interface (UI):** OS provides CLI (Command Line), GUI (Graphical), or Touch-screen interfaces.
- **Program Execution:** OS loads a program into memory, runs it, and ends it (normally or with an error).
- **I/O Operations:** Programs often need to read/write files or use devices. OS handles this on their behalf.
- **File System Manipulation:** Programs can create, delete, read, write, search files and directories, and manage permissions.
- **Communications:** Processes share data via shared memory or message passing, on the same machine or over a network.
- **Error Detection:** OS constantly checks for errors in CPU, memory, I/O devices, and user programs, taking appropriate action.

4. Category B: System Efficiency Services

- **Resource Allocation:** When multiple jobs run at once, OS allocates CPU time, memory, and devices to each.
- **Accounting:** OS tracks which users use what resources and how much.
- **Protection and Security:** OS controls access to resources (protection) and defends against external threats (security) through user authentication.

5. Diagram: View of OS Services



6. Practical Example

Real-life: When you open a file in Notepad (Windows), the OS handles the I/O operation, checks file permissions (protection), and if the file is missing, detects the error and shows you a message.

System: On Linux, running 'ls' uses file-system manipulation. Running a Python script uses program execution. Both are OS services.

7. Key Points

- OS services = environment for programs + services for users.
- 6 user services: UI, Program Execution, I/O, File System, Communication, Error Detection.
- 3 system services: Resource Allocation, Accounting, Protection & Security.
- **System Calls** are how programs request OS services.
- Security = external defense; Protection = internal resource control.

8. Short Revision Notes

OS services provide programs and users with the tools to run correctly and efficiently. User-facing services include UI, I/O, file handling, error detection, and communications. System-efficiency services include resource allocation, accounting, and protection/security. All services are accessed through System Calls.

Topic 3: System Calls

1. Definition

System calls are the **programmatic interface** through which user programs request services from the Operating System kernel.

They act as a controlled gateway from **User Mode** (unprivileged) to **Kernel Mode** (privileged), where hardware can be accessed.

Most system calls are available as functions written in **C** or **C++**; very low-level ones may use assembly language.

2. Why System Calls Are Needed

OS services (like reading a file) can only be executed when the CPU is in **Supervisor/Kernel Mode**.

User programs run in **User Mode** and cannot access hardware directly.

System calls provide the **safe, controlled bridge** between the two modes using **software interrupts**.

3. Working Process (Step-by-Step)

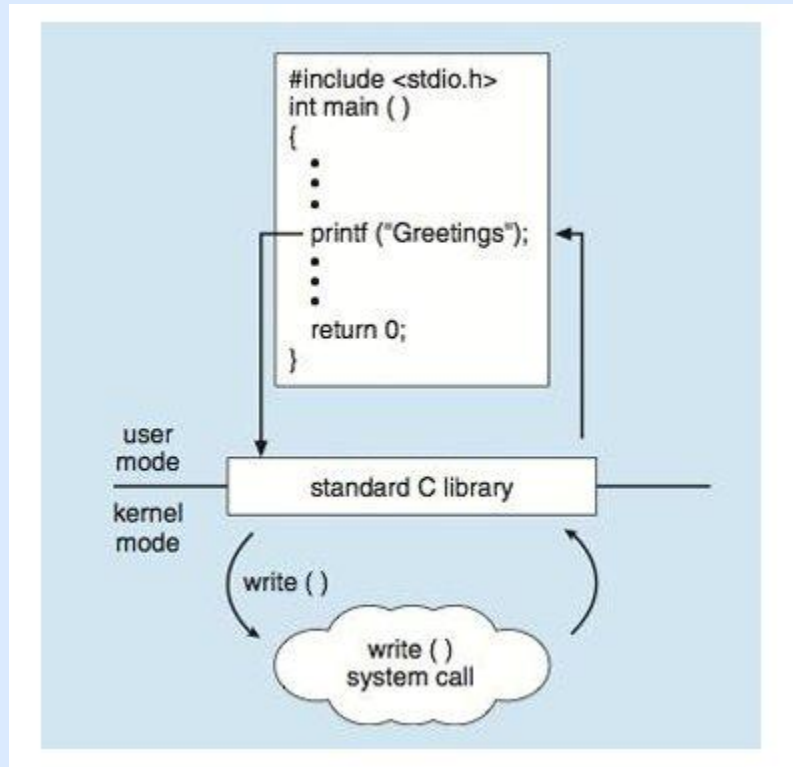
7. A user program calls an API function (e.g., `open()`, `read()`).
8. The API function triggers a **software interrupt** (trap instruction).
9. The CPU switches from User Mode to **Kernel Mode**.
10. The **system-call interface** intercepts the call and looks up a table indexed by system call number.
11. The correct OS kernel routine executes (e.g., file read operation).
12. The kernel returns **status and return value** to the calling program.
13. CPU switches back to **User Mode**.

4. Diagram: API – System Call – OS Relationship


```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count)
```

return value function name parameters



5. Categories of System Calls

Category	Windows	Unix/Linux
Process Control	CreateProcess(), ExitProcess()	fork(), exit(), wait()
File Management	CreateFile(), ReadFile(), WriteFile()	open(), read(), write(), close()
Device Management	SetConsoleMode(), ReadConsole()	ioctl(), read(), write()

Information	GetCurrentProcessID(), SetTimer()	getpid(), alarm(), sleep()
Communications	CreatePipe(), CreateFileMapping()	pipe(), shm_open(), mmap()
Protection	SetFileSecurity()	chmod(), umask(), chown()

Examples of Windows and Unix System Calls

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS

The following illustrates various equivalent system calls for Windows and UNIX operating systems.

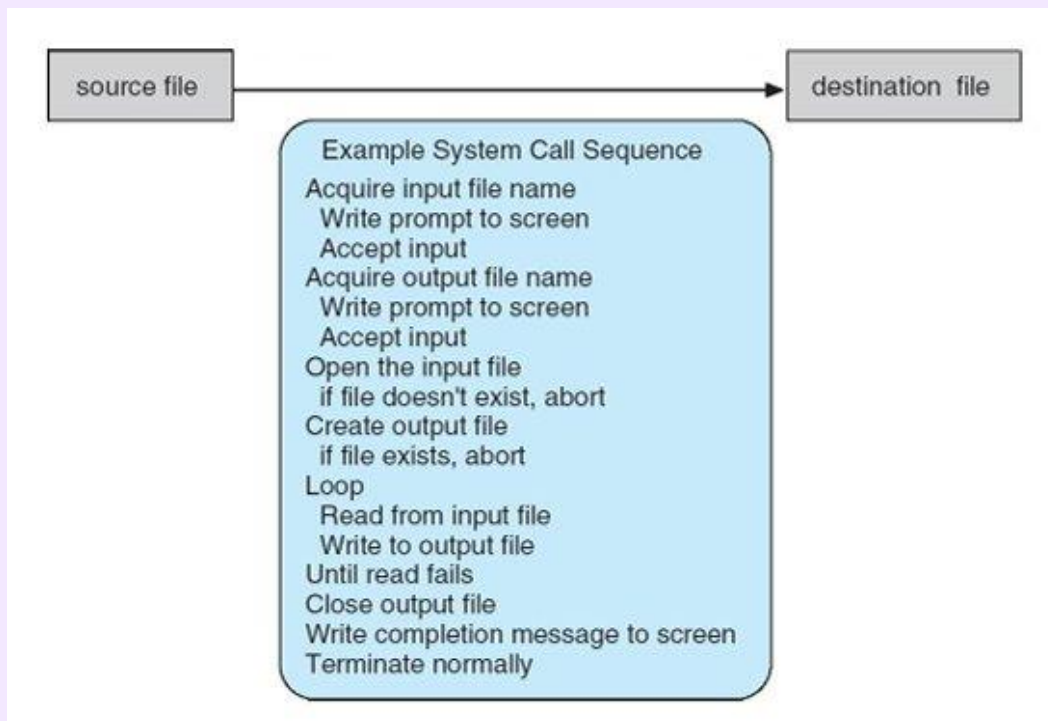
	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

6. Practical Example

Copying a file (e.g., `cp source.txt dest.txt` in Linux):

14. Acquire input filename → write prompt → accept input.
15. Acquire output filename → write prompt → accept input.
16. Open input file (if not found, abort).
17. Create output file (if exists, abort).
18. Loop: Read from input → Write to output → Until read fails.
19. Close output file. Write completion message. Terminate normally.

Each of these steps invokes multiple system calls.



7. Key Points

- System calls = interface between user programs and OS kernel.
- **Software interrupts** trigger mode switch from User Mode to Kernel Mode.
- OS maintains a **system call table** indexed by call number.
- Programmer uses **API** (Win32, POSIX, Java API) and never directly invokes system calls.
- On Linux/C, the library **libc** provides access to POSIX API.

8. API vs System Call

API	System Call
High-level, programmer-friendly interface.	Low-level interface to OS kernel.
Example: printf(), fopen() in C.	Example: write(), open() in Linux kernel.
Runs in User Mode.	Executes in Kernel Mode.
Provided by libraries (e.g., libc).	Built into the OS kernel.

9. Short Revision Notes

System calls are the gateway from User Mode programs to Kernel Mode OS services. They are triggered by software interrupts. The OS maintains a numbered table of system calls. Programmers use APIs (Win32, POSIX) which internally invoke system calls. Six categories: process control, file management, device management, information, communications, protection.

Topic 4: OS Design Structures

1. Definition

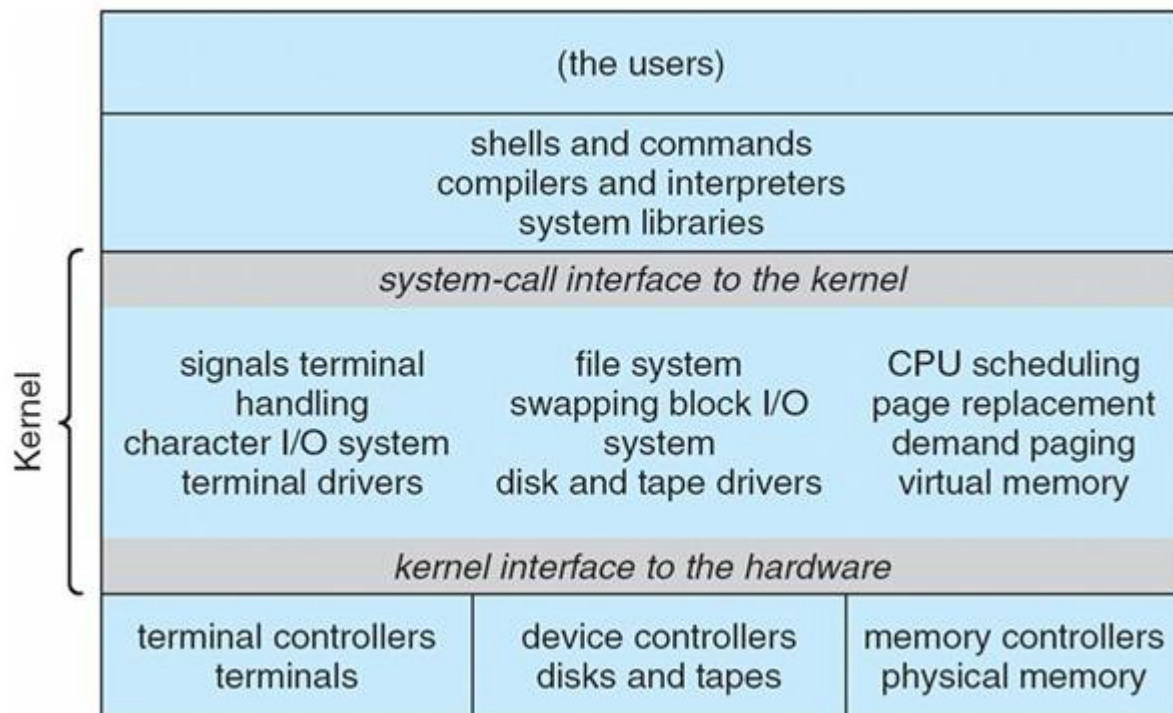
OS structure refers to the **way the OS is internally organized** into components and how those components interact.

There are several common approaches: Simple (Monolithic), Layered, Microkernel, Modular, and Hybrid.

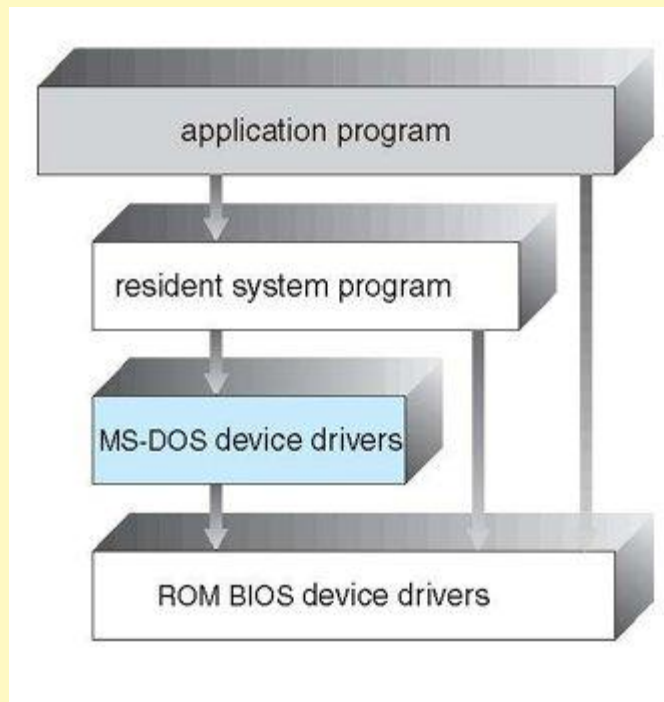
2A. Simple Structure (Monolithic) — MS-DOS

MS-DOS was designed to provide maximum functionality in minimum space. It has **no clear separation** between layers.

- Single-tasking: only one program runs at a time.
- No process is created; the program is loaded directly into memory, overwriting everything except the kernel.
- When the program exits, the shell reloads.



- Layers: Application Program → Resident System Program → MS-DOS Device Drivers → ROM BIOS Device Drivers



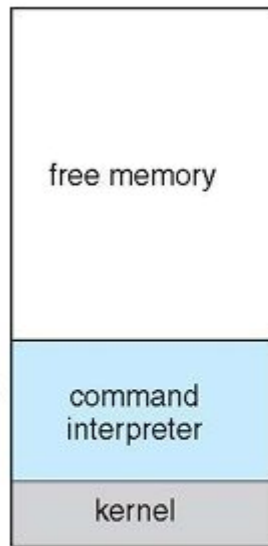
Drawback: All components can access each other, making it insecure and unstable.

2B. UNIX Structure

UNIX has two main parts: **System Programs** and the **Kernel**.

The kernel sits between the system-call interface and the hardware. It handles CPU scheduling, memory management, file systems, and more.

UNIX is **beyond simple but not fully layered** — it has some structure but the kernel handles too many functions for one level.

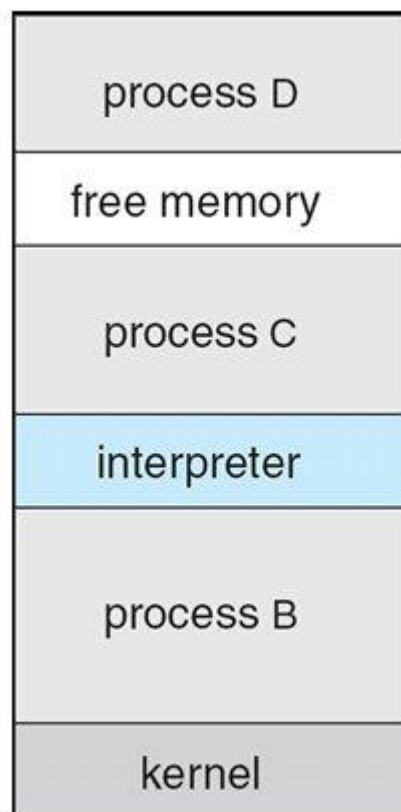


(a)



(b)

) At system startup (b) running a program



2C. Layered Approach

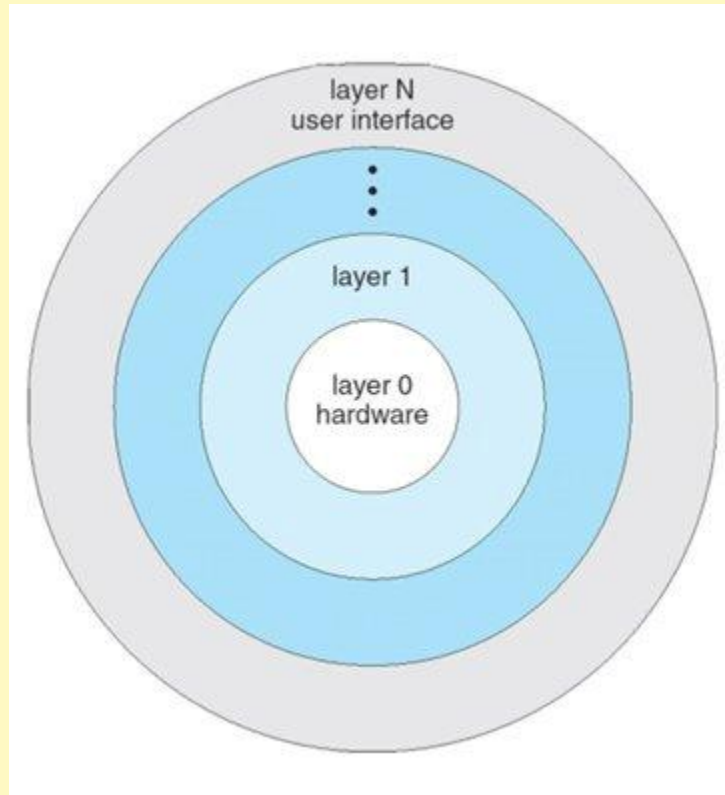
The OS is divided into **N layers**. Layer 0 = hardware. Layer N = user interface.

Each layer only uses functions from the **layer directly below it**. This ensures modularity.

Advantage: Easy to debug and verify each layer independently.

Disadvantage: Performance overhead; careful layer ordering is difficult.

Diagram: [Layer N: User Interface] → ... → [Layer 1] → [Layer 0: Hardware]



2D. Microkernel Structure

The microkernel approach removes **all non-essential components** from the kernel and makes them user-level programs.

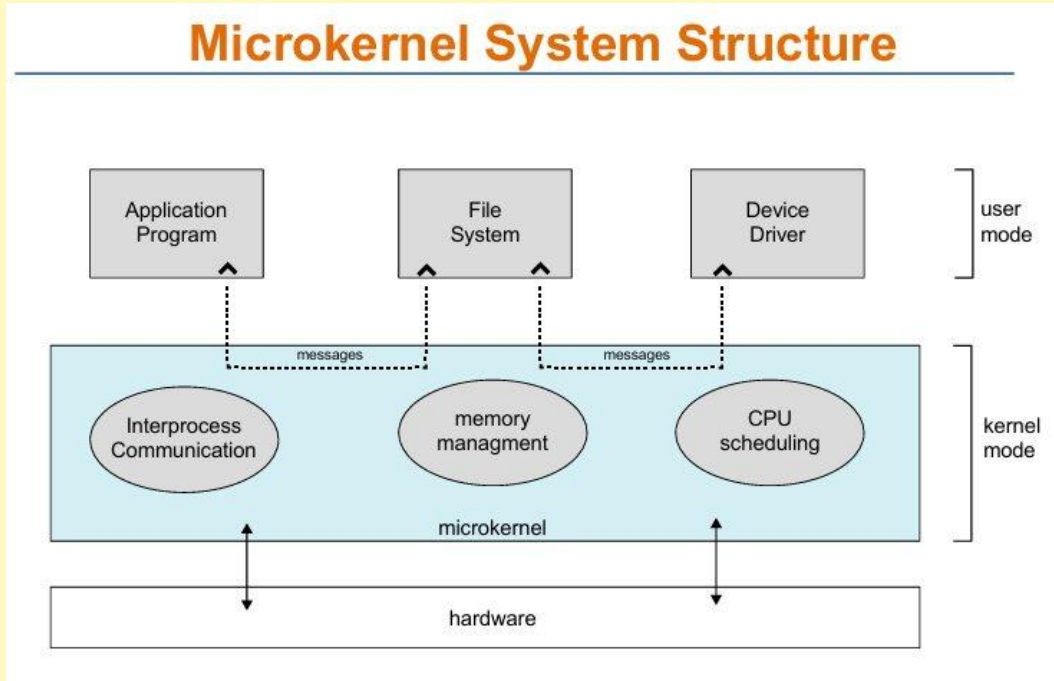
The tiny kernel handles only: **inter-process communication, basic memory management, CPU scheduling**.

All other services (file system, device drivers) run in **User Mode** as separate processes.

- **Advantage — Reliable:** If a service crashes, only that service is affected, not the whole OS.
- **Advantage — Secure:** User-mode services have no supervisor power.

- **Advantage — Portable:** Easier to port to new hardware architectures.
- **Disadvantage:** Performance overhead due to message passing between user space and kernel space.

Example: **Mach** microkernel (base for macOS Darwin). Also: QNX, L4, Minix.

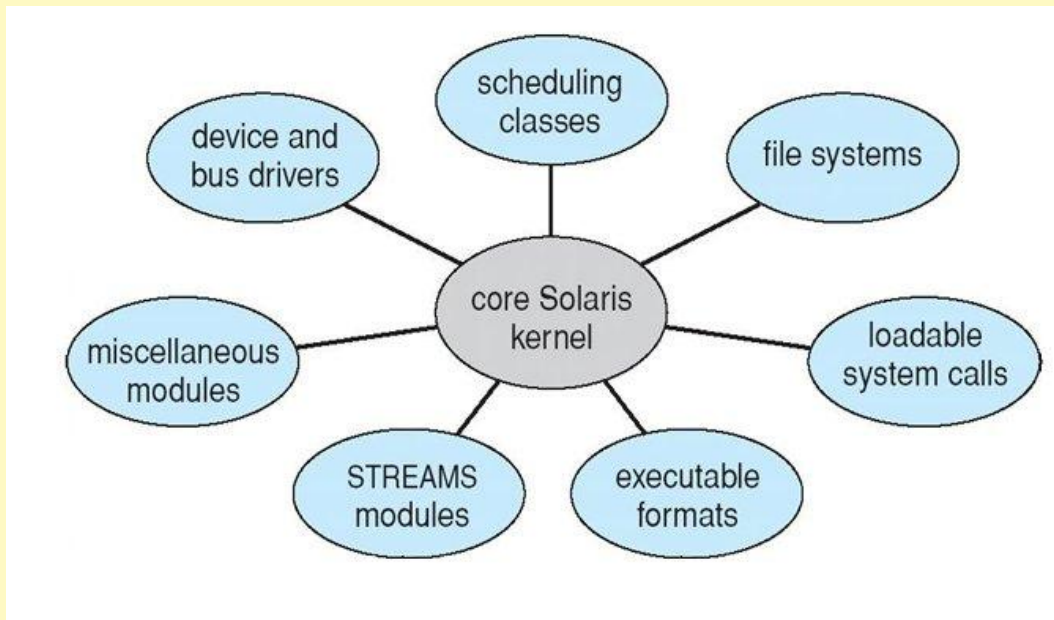


2E. Modular (Loadable Kernel Modules)

Most modern OSes implement **Loadable Kernel Modules (LKMs)**.

- Each module is an independent component that can be **loaded or unloaded at runtime**.
- Components communicate through well-defined interfaces.
- Similar to the layered approach but **more flexible**: modules can call any other module.

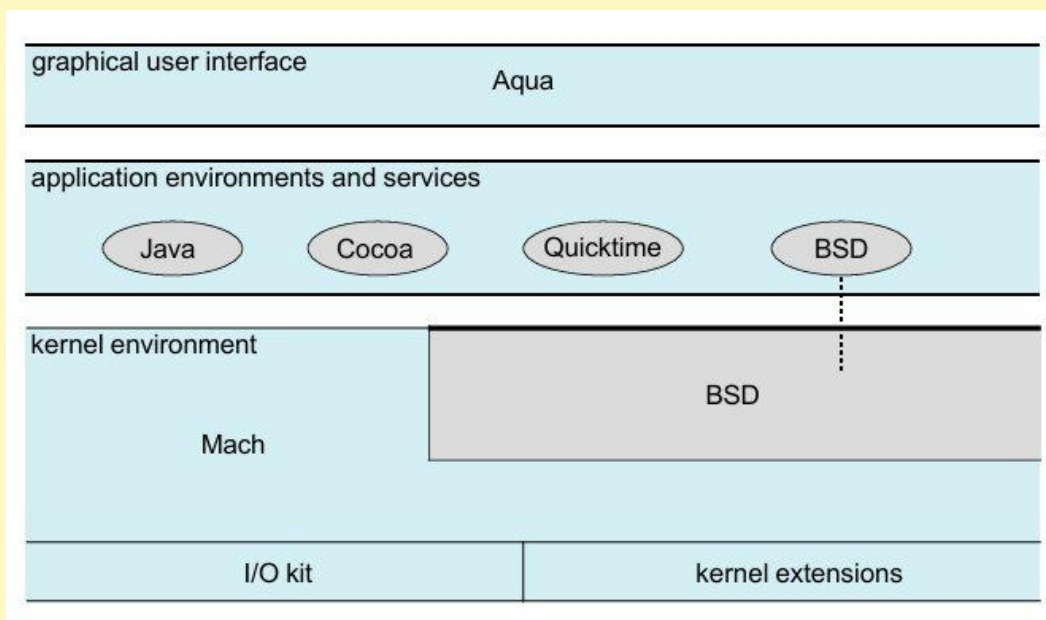
Examples: **Linux, Solaris** — add new file system support or device drivers without rebooting.



2F. Hybrid Systems

Most real-world OSes combine multiple approaches to balance performance, security, and usability.

- **Linux:** Monolithic kernel + loadable modules for dynamic functionality.
- **Windows:** Mostly monolithic + microkernel for subsystem personalities.
- **macOS/iOS:** Hybrid layered; Aqua GUI on top, then Cocoa/Java/BSD, then Mach microkernel + BSD Unix kernel + I/O Kit.



- **Android:** Based on modified Linux kernel + Dalvik VM + Java API. Adds power management.

3. Diagram: Microkernel vs Monolithic

Monolithic:

[App] → [File System + Device Driver + Scheduler + Memory] (all in kernel) → [Hardware]

Microkernel:

[App] ←messages→ [File System (user)] ←messages→ [Device Driver (user)]

↑ messages

[Microkernel: IPC + Basic Memory + CPU Scheduling] → [Hardware]

4. Comparison Table

Feature	Monolithic	Microkernel
Size	Large (millions of lines)	Tiny (thousands of lines)
Speed	Faster (direct calls)	Slower (message overhead)
Stability	One crash kills the OS	Only the service crashes
Security	Large attack surface	Smaller, more secure core
Examples	Linux, Windows, macOS	QNX, L4, Minix, Mach

5. Practical Example

Real-life: Your Android phone uses a modified Linux kernel (monolithic + modular). The Dalvik VM runs your apps. If an app crashes, the kernel continues running; only the app process dies.

System: On Linux, you can load a new file system driver at runtime without rebooting: 'modprobe ext4'. This is the modular approach in action.

6. Key Points

- **Simple/Monolithic:** No clear separation. Fast but insecure. Example: MS-DOS, early UNIX.
- **Layered:** Clean separation but slower. Each layer only uses the one below.
- **Microkernel:** Tiny kernel, services in User Mode. Secure but slower.

- **Modular:** Dynamic loading of kernel components. Flexible. Example: Linux.
- **Hybrid:** Real-world OSes mix approaches. Example: macOS, Windows, Android.

7. Short Revision Notes

OS structure defines how the OS is organized internally. Simple/monolithic = everything in kernel (fast, risky). Layered = structured but inflexible. Microkernel = small kernel, services in user mode (secure, slower). Modular = loadable components (Linux). Hybrid = real OSes combine all approaches. Most modern OSes (Linux, Windows, macOS, Android) are hybrid.

Topic 5: Mobile Operating Systems — iOS and Android

1. Definition

Mobile operating systems are designed specifically for **smartphones and tablets**. They are optimized for **touch input, battery life, limited memory, and ARM processor architectures**.

The two dominant mobile OSes are **iOS** (Apple) and **Android** (Google/Open Handset Alliance).

2. iOS

Developer: Apple Inc. Used on iPhone and iPad.

- Structured on **Mac OS X**, with added mobile functionality.
- Does **not** run OS X desktop applications natively.
- Runs on **ARM processors** (not Intel).
- **Cocoa Touch:** Objective-C API used to develop iOS apps.
- **Media Services:** Layer for graphics, audio, and video.
- **Core Services:** Provides cloud computing and database access.
- **Core OS:** Based on Mac OS X kernel (Mach + BSD).

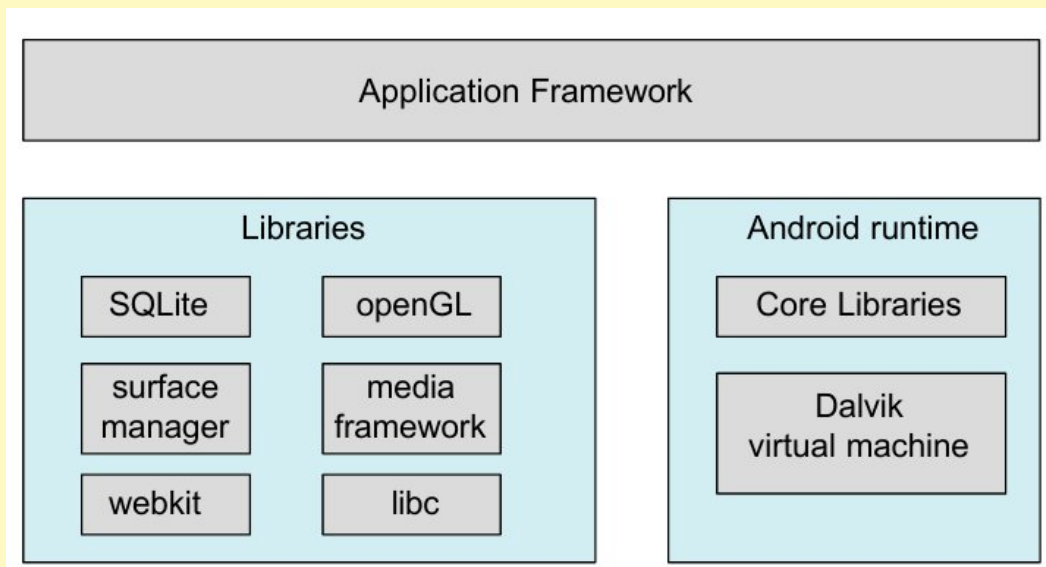
Layers (top to bottom): Cocoa Touch → Media Services → Core Services → Core OS

3. Android

Developer: Open Handset Alliance (primarily Google). Open Source.

- Based on a **modified Linux kernel**, with added **power management**.
- Linux kernel provides **process management, memory management, device drivers**.
- **Dalvik VM:** Android's custom virtual machine. Apps are written in Java, compiled to Java bytecode, then translated to Dalvik executable (.dex) format.
- **Core Libraries:** Include SQLite (database), WebKit (browser), OpenGL, libc.
- **Application Framework:** Provides services to apps (activity manager, notifications, content providers).

Architecture (top to bottom): Application Framework → Android Runtime (Core Libs + Dalvik VM) → Libraries + Linux Kernel



4. Comparison Table: iOS vs Android

iOS	Android
Closed source (Apple only).	Open Source (any manufacturer).
Runs on Apple ARM hardware.	Runs on various ARM hardware.
Apps in Objective-C or Swift.	Apps in Java (or Kotlin).
Based on Mach + BSD (Mac OS X kernel).	Based on modified Linux kernel.
Cocoa Touch API.	Android API + Dalvik VM.
Stricter app store (App Store).	More open app distribution (Play Store + sideloading).

5. Practical Example

Real-life: When you open the Pathao app on Android, the Android runtime (Dalvik VM) runs the Java bytecode. The Linux kernel manages GPS hardware access, network connection, and battery. The Application Framework handles the map display and location services.

iOS: When you use bKash on an iPhone, Core Services handles the network requests. Core OS manages security (Secure Enclave for payment). Media Services renders the UI animations.

6. Key Points

- **iOS** is closed, Apple-only, built on macOS kernel (Mach + BSD), uses ARM.
- **Android** is open-source, built on Linux kernel, uses Dalvik VM.
- Android apps are written in **Java**, compiled to **.dex** for Dalvik VM.
- Both support touch UI, media, and cloud services.
- Both are **hybrid** OS architectures (layered + microkernel elements).

7. Short Revision Notes

iOS is Apple's mobile OS built on Mac OS X (Mach + BSD). Its layers are Cocoa Touch, Media Services, Core Services, Core OS. Android is Google's open-source mobile OS based on a modified Linux kernel. Apps run on the Dalvik VM. Both are hybrid systems optimized for ARM hardware, touch input, and limited resources.